In [1]:
```
from google import genai

client = genai.Client(api_key="I deleted my API key. Place your owned API key here")

# In ra danh sách các model có sẵn
for m in client.models.list():
    if "generateContent" in m.supported_actions:
        print(m.name)
```

```
models/gemini-2.5-flash
models/gemini-2.5-pro
models/gemini-2.0-flash
models/gemini-2.0-flash-001
models/gemini-2.0-flash-lite-001
models/gemini-2.0-flash-lite
models/gemini-2.5-flash-preview-tts
models/gemini-2.5-pro-preview-tts
models/gemma-4-26b-a4b-it
models/gemma-4-31b-it
models/gemini-flash-latest
models/gemini-flash-lite-latest
models/gemini-pro-latest
models/gemini-2.5-flash-lite
models/gemini-2.5-flash-image
models/gemini-3-pro-preview
models/gemini-3-flash-preview
models/gemini-3.1-pro-preview
models/gemini-3.1-pro-preview-customtools
models/gemini-3.1-flash-lite-preview
models/gemini-3.1-flash-lite
models/gemini-3-pro-image-preview
models/gemini-3-pro-image
models/nano-banana-pro-preview
models/gemini-3.1-flash-image-preview
models/gemini-3.1-flash-image
models/gemini-3.1-flash-lite-image
models/gemini-3.5-flash
models/gemini-3.5-flash-lite
models/gemini-omni-flash-preview
models/gemini-3.6-flash
models/lyria-3-clip-preview
models/lyria-3-pro-preview
models/gemini-3.1-flash-tts-preview
models/gemini-robotics-er-1.5-preview
models/gemini-robotics-er-1.6-preview
models/gemini-robotics-er-2-preview
models/gemini-2.5-computer-use-preview-10-2025
models/antigravity-preview-05-2026
models/deep-research-max-preview-04-2026
models/deep-research-preview-04-2026
models/deep-research-pro-preview-12-2025
```

In [2]:
```
import json
import asyncio
import os
import pandas as pd

from google import genai
from google.genai import types

from bert_score import score
```

```python
from rouge_score import rouge_scorer


# ======================================================
# CONFIG
# ======================================================

GEMINI_API_KEY = os.getenv(
    "GEMINI_API_KEY",
    “I deleted my API key. Place your owned API key here”
)

MODEL_NAME = "gemini-3.6-flash"

TEST_FILE = "./Training_Data/test.json"

CHECKPOINT_CSV = (
    "./Training_Data/"
    "gemini_predictions_checkpoint.csv"
)

OUTPUT_CSV = (
    "./Training_Data/"
    "Baseline_gemini-3.6-flash_predictions.csv"
)


# nghỉ giữa các request
DELAY_AFTER_RESPONSE = 4.5


client = genai.Client(
    api_key=GEMINI_API_KEY
)


# ======================================================
# LOAD DATASET
# ======================================================

dataset = []

with open(TEST_FILE, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()

        if line:
            dataset.append(
                json.loads(line)
            )


print(
    f"Loaded {len(dataset)} samples"
)



# ======================================================
# GEMINI REQUEST
```

```
# KHÔNG BỎ QUA REQUEST
# ==================================================

async def send_and_wait_for_response(
        sample,
        idx,
        total
):

    prompt = f"""
Instruction:
{sample['instruction']}

Input:
{sample['input']}

Answer:
"""


    attempt = 1


    while True:

        try:

            print(
                f"\n[{idx+1}/{total}] "
                f"Sending Gemini request "
                f"(attempt {attempt})"
            )


            response = await client.aio.models.generate_content(
                model=MODEL_NAME,
                contents=prompt,
                config=types.GenerateContentConfig(
                    temperature=0.0
                )
            )


            if response is None:
                raise RuntimeError(
                    "Empty Gemini response object"
                )


            prediction = (
                response.text.strip()
                if response.text
                else ""
            )


            if prediction == "":
                raise RuntimeError(
                    "Gem" \
                    "ini returned empty text"
```

```
                )


            print(
                "✅ Response received"
            )

            print(
                prediction[:120],
                "..."
            )


            return prediction



        except Exception as e:
            error_message = str(e)
            print(
                "❌ Gemini error:",
                e
            )

            # ----------------------------------------
            # 429: Quota exceeded -> DỪNG CHƯƠNG TRÌNH
            # ----------------------------------------
            if "429" in error_message or "RESOURCE_EXHAUSTED" in
error_message:

                print("\n========================================")
                print("❌ Gemini quota exceeded (429).")
                print("⛔ Program stopped.")
                print("📌 Checkpoint has been preserved.")
                print("▶ Run the program again after quota resets.")
                print("========================================\n")

                raise SystemExit("Stopped because Gemini quota was
exceeded.")

            # ----------------------------------------
            # Các lỗi khác -> Retry
            # ----------------------------------------


            wait_time = min(
                60,
                attempt * 5
            )


            print(
                f"Waiting {wait_time}s before retry..."
            )


            await asyncio.sleep(
                wait_time
            )
```

```
            attempt += 1



# ======================================================
# MAIN
# ======================================================

async def main():


    predictions = []
    references = []

    start_idx = 0



    # ----------------------------------------------
    # RESUME CHECKPOINT
    # ----------------------------------------------

    if os.path.exists(CHECKPOINT_CSV):

        print(
            "Checkpoint found"
        )

        checkpoint_df = pd.read_csv(
            CHECKPOINT_CSV
        )


        checkpoint_df["prediction"] = (
            checkpoint_df["prediction"]
            .fillna("")
        )


        predictions = (
            checkpoint_df["prediction"]
            .tolist()
        )


        references = (
            checkpoint_df["ground_truth"]
            .tolist()
        )


        start_idx = len(predictions)


        print(
            f"Resume from sample "
            f"{start_idx+1}"
        )
```

```
else:

    print(
        "No checkpoint. Start from beginning."
    )



total = len(dataset)



# ------------------------------------------------
# GEMINI LOOP
# ------------------------------------------------

for idx in range(
    start_idx,
    total
):


    sample = dataset[idx]


    prediction = await send_and_wait_for_response(
        sample,
        idx,
        total
    )


    # chỉ append khi có response hợp lệ

    predictions.append(
        prediction
    )

    references.append(
        sample["output"]
    )



    # ----------------------------
    # SAVE CHECKPOINT
    # ----------------------------

    if (
        len(predictions) % 2 == 0
        or idx == total - 1
    ):


        checkpoint = pd.DataFrame(
            {
                "instruction":
                [
                    x["instruction"]
```

```
                    for x in dataset[:len(predictions)]
                ],

                "input":
                [
                    x["input"]
                    for x in dataset[:len(predictions)]
                ],

                "ground_truth":
                references,

                "prediction":
                predictions
            }
        )


        checkpoint.to_csv(
            CHECKPOINT_CSV,
            index=False,
            encoding="utf-8"
        )


        print(
            f"💾 Saved checkpoint "
            f"{len(predictions)}/{total}"
        )



    # ----------------------------
    # WAIT BEFORE NEXT REQUEST
    # ----------------------------

    if idx < total - 1:

        print(
            f"Waiting {DELAY_AFTER_RESPONSE}s..."
        )

        await asyncio.sleep(
            DELAY_AFTER_RESPONSE
        )



# ================================================
# AFTER ALL GEMINI RESPONSES
# START EVALUATION
# ================================================


print(
    "\n================================"
)

print(
    "All Gemini responses completed."
```

```
    )

    print(
        "Computing BERTScore..."
    )



    assert len(predictions) == len(references)


    # ------------------------------------------------
    # BERT SCORE
    # ------------------------------------------------

    P, R, F1 = score(
        predictions,
        references,
        lang="en",
        verbose=True
    )



    # ------------------------------------------------
    # ROUGE-L
    # ------------------------------------------------

    print(
        "Computing ROUGE-L..."
    )


    rouge = rouge_scorer.RougeScorer(
        [
            "rougeL"
        ],
        use_stemmer=True
    )


    rouge_precision = []
    rouge_recall = []
    rouge_f1 = []



    for pred, ref in zip(
        predictions,
        references
    ):


        result = rouge.score(
            ref,
            pred
        )


        score_value = (
            result["rougeL"]
```

```
        )


        rouge_precision.append(
            score_value.precision
        )

        rouge_recall.append(
            score_value.recall
        )

        rouge_f1.append(
            score_value.fmeasure
        )



    # ------------------------------------------------
    # SAVE FINAL RESULT
    # ------------------------------------------------

    final_df = pd.DataFrame(
        {

            "instruction":
            [
                x["instruction"]
                for x in dataset
            ],

            "input":
            [
                x["input"]
                for x in dataset
            ],


            "ground_truth":
            references,


            "prediction":
            predictions,


            "bert_precision":
            P.numpy(),


            "bert_recall":
            R.numpy(),


            "bert_f1":
            F1.numpy(),



            "rougeL_precision":
            rouge_precision,
```

```
            "rougeL_recall":
            rouge_recall,


            "rougeL_f1":
            rouge_f1

        }
    )



    final_df.to_csv(
        OUTPUT_CSV,
        index=False,
        encoding="utf-8"
    )



    # ------------------------------------------------
    # REPORT
    # ------------------------------------------------

    print("\n==============================")
    print("Average BERTScore")

    print(
        f"Precision: {P.mean():.4f}"
    )

    print(
        f"Recall   : {R.mean():.4f}"
    )

    print(
        f"F1       : {F1.mean():.4f}"
    )



    print("\n==============================")
    print("Average ROUGE-L")


    print(
        f"Precision: "
        f"{sum(rouge_precision)/len(rouge_precision):.4f}"
    )


    print(
        f"Recall   : "
        f"{sum(rouge_recall)/len(rouge_recall):.4f}"
    )


    print(
        f"F1       : "
        f"{sum(rouge_f1)/len(rouge_f1):.4f}"
```

```
    )


    print(
        "\nSaved:",
        OUTPUT_CSV
    )



# ================================================
# RUN IN JUPYTER
# ================================================

await main()
```

```
/opt/anaconda3/envs/LLM/lib/python3.11/site-packages/tqdm/auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Loaded 121 samples
Checkpoint found
Resume from sample 109

[109/121] Sending Gemini request (attempt 1)
✅ Response received
**Yes, itemset [3259] is a frequent itemset (FI).**

### **Reasoning:**
1. **Definition of Frequent Itemset:** An itemse ...
Waiting 4.5s...

[110/121] Sending Gemini request (attempt 1)
✅ Response received
Based on the provided input data, here is the analysis for the itemset **[39,
48, 1807]**:

---

### **1. Analysis of It ...
💾 Saved checkpoint 110/121
Waiting 4.5s...

[111/121] Sending Gemini request (attempt 1)
✅ Response received
Based on the provided metrics, **Itemset A [39, 48, 8451] has a higher
business impact than Itemset B [39, 41, 4334]**.  ...
Waiting 4.5s...

[112/121] Sending Gemini request (attempt 1)
✅ Response received
Here is the comparison between Itemset A **[39, 4292]** and Itemset B **[48,
4095]** across the specified metrics:

---
 ...
💾 Saved checkpoint 112/121
Waiting 4.5s...
```

[113/121] Sending Gemini request (attempt 1)
✅ Response received
Yes, the itemset **[39, 48, 306]** is a **frequent itemset (FI)**.

Explanation:
1. **Support vs. Support Threshold ...
Waiting 4.5s...

[114/121] Sending Gemini request (attempt 1)
✅ Response received
Here is a comparative analysis of itemset **[6447]** (Itemset A) and itemset **[2490]** (Itemset B) based on the request ...
💾 Saved checkpoint 114/121
Waiting 4.5s...

[115/121] Sending Gemini request (attempt 1)
✅ Response received
Based on the provided data, here is the analysis of itemset **[604]**:

1. Metric Analysis

* **Number of Suppor ...
Waiting 4.5s...

[116/121] Sending Gemini request (attempt 1)
✅ Response received
Here is a comparative analysis of **Itemset A [41, 11610]** and **Itemset B [39, 5294]** based on the specified metrics: ...
💾 Saved checkpoint 116/121
Waiting 4.5s...

[117/121] Sending Gemini request (attempt 1)
✅ Response received
Based on the provided metrics, **Itemset [609] generally outperforms Itemset [9435]** in overall frequency and FI score, ...
Waiting 4.5s...

[118/121] Sending Gemini request (attempt 1)
✅ Response received
Based on the provided data, here is the performance comparison between **Itemset A [41, 994]** and **Itemset B [39, 4084 ...
💾 Saved checkpoint 118/121
Waiting 4.5s...

[119/121] Sending Gemini request (attempt 1)
✅ Response received
To determine whether the itemset **[39, 291]** satisfies the Frequent Itemset (FI) criteria, we compare its support metr ...
Waiting 4.5s...

[120/121] Sending Gemini request (attempt 1)
✅ Response received
To determine if the itemset **[39, 48, 5439]** satisfies the Frequent Itemset (FI) criteria, we compare its support metr ...
💾 Saved checkpoint 120/121
Waiting 4.5s...

```
[121/121] Sending Gemini request (attempt 1)
✅ Response received
Based on the provided metrics, itemset **[48, 41, 2996]** qualifies as a
frequent itemset (FI) for the following key rea ...
💾 Saved checkpoint 121/121

================================
All Gemini responses completed.
Computing BERTScore...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 389/389 [00:00<00:00, 7613.23it/s]
[transformers] RobertaModel LOAD REPORT from: roberta-large
Key                         | Status     |
----------------------------+------------+-
lm_head.layer_norm.weight | UNEXPECTED |
lm_head.dense.weight      | UNEXPECTED |
lm_head.layer_norm.bias   | UNEXPECTED |
lm_head.dense.bias        | UNEXPECTED |
lm_head.bias              | UNEXPECTED |
pooler.dense.weight       | MISSING    |
pooler.dense.bias         | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
not ok if you expect identical arch.
- MISSING:      those params were newly initialized because missing from the
checkpoint. Consider training on your downstream task.
calculating scores...
computing bert embedding.
100%|██████████| 4/4 [00:20<00:00,  5.14s/it]
computing greedy matching.
100%|██████████| 2/2 [00:00<00:00, 29.35it/s]
done in 20.65 seconds, 5.86 sentences/sec
Computing ROUGE-L...

==============================
Average BERTScore
Precision: 0.8125
Recall   : 0.8617
F1       : 0.8362

==============================
Average ROUGE-L
Precision: 0.1807
Recall   : 0.3897
F1       : 0.2311

Saved: ./Training_Data/Baseline_gemini-3.6-flash_predictions.csv
```

** Numerical Faithfulness **

In [12]:

```python
import re
import pandas as pd

import pandas as pd

df_csv = pd.read_csv(OUTPUT_CSV)
```

```python
# ============================================================
# Extract numbers
# ============================================================

def extract_numbers(text):
    """
    Extract all integers and decimal numbers from text.
    """
    return re.findall(r"\d+(?:\.\d+)?", text)


# ============================================================
# Numerical Faithfulness
# ============================================================

def numerical_faithfulness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)
    prediction_numbers = extract_numbers(prediction_text)

    # Prediction không sinh số
    if len(prediction_numbers) == 0:
        return {
            "InputNumbers": input_numbers,
            "PredictionNumbers": prediction_numbers,
            "HallucinatedNumbers": [],
            "CorrectNumbers": 0,
            "PredictionCount": 0,
            "NumericalFaithfulness": 1.0
        }

    correct = 0
    hallucinated = []

    for num in prediction_numbers:

        if num in input_numbers:
            correct += 1
        else:
            hallucinated.append(num)

    score = correct / len(prediction_numbers)

    return {
        "InputNumbers": input_numbers,
        "PredictionNumbers": prediction_numbers,
        "HallucinatedNumbers": hallucinated,
        "CorrectNumbers": correct,
        "PredictionCount": len(prediction_numbers),
        "NumericalFaithfulness": score
    }


# ============================================================
# Evaluation
# ============================================================

results = []
```

```
for input_text, prediction in df_csv[
    ["input", "prediction"]
].itertuples(index=False):

    result = numerical_faithfulness(
        input_text,
        prediction
    )

    results.append(result)



df = pd.DataFrame(results)

print("=" * 60)
print("Average Numerical Faithfulness:",
      df["NumericalFaithfulness"].mean())
print("=" * 60)

df.to_csv(
    "NumericalFaithfulness_Gemini-3.6-flash.csv",
    index=False,
    encoding="utf-8-sig"
)
```

```
============================================================
Average Numerical Faithfulness: 0.5308933828490727
============================================================
```

** Explanation Completeness **

In [20]:
```
import re

# ============================================================
# Keywords
# ============================================================

SUPPOR_KEYWORDS = [
    "support",
    "supports",
    "support threshold",
    "support exceeds",
    "purchased frequently",
    "frequently occurring",
    "frequent itemset"
]

AVG_LENGTH_KEYWORDS = [
    "average support-transaction length",
    "average transaction length",
    "longer transaction",
    "shorter transaction",
    "longer transactions",
    "shorter transactions",
    "average length"
]

FI_SCORE_KEYWORDS = [

    "fi score",

    "score",
```

```
    "overall performance",
    "better performance",
    "higher score",
    "lower score",
    "overall score",
    "business impact",
    "overall impact"
]


# ============================================================
# Utility
# ============================================================

def contains_keywords(text, keywords):
    text = text.lower()
    return any(keyword.lower() in text for keyword in keywords)


# ============================================================
# Support
# ============================================================

def evaluate_support(prediction):
    return int(
        contains_keywords(
            prediction,
            SUPPOR_KEYWORDS
        )
    )


# ============================================================
# Average Support Length
# ============================================================

def evaluate_average_length(prediction):
    return int(
        contains_keywords(
            prediction,
            AVG_LENGTH_KEYWORDS
        )
    )


# ============================================================
# FI Score
# ============================================================
```

```python
def evaluate_fi_score(prediction):

    return int(
        contains_keywords(
            prediction,
            FI_SCORE_KEYWORDS
        )
    )


# ============================================================
# Overall
# ============================================================

def explanation_completeness(prediction):

    support = evaluate_support(prediction)

    avg = evaluate_average_length(prediction)

    fi = evaluate_fi_score(prediction)

    return {
        "support": support,
        "average_length": avg,
        "fi_score": fi,
        "ExplanationCompleteness":
            (support + avg + fi) / 3
    }


import pandas as pd

rows = []

for sample in df_csv.to_dict(orient="records"):

    rows.append(
        explanation_completeness(
            sample["prediction"]
        )
    )

df = pd.DataFrame(rows)

# print(df.head())

# print()

print("="*50)
print(df.mean())

df.to_csv(
    "./Training_Data/ExplanationCompleteness_Gemini-3.6-flash.csv",
    index=False
)
```

```
==================================================
support                    1.000000
average_length             0.760331
fi_score                   0.677686
ExplanationCompleteness    0.812672
```

```
dtype: float64
```

** Numerical Correctness **

In [21]:

```python
import re
import pandas as pd

# ============================================================
# CONFIG
# ============================================================

TOLERANCE = 0.1

# ============================================================
# Remove Itemset IDs
# ============================================================

def remove_itemsets(text):
    """
    Remove itemset IDs.

    Example:
        [39, 41, 1799]
        [6009]
    """

    if text is None:
        return ""

    return re.sub(r"\[[^\]]*\]", "", text)


# ============================================================
# Extract Numbers
# ============================================================

def extract_numbers(text):
    """
    Extract all numerical values.
    """

    text = remove_itemsets(text)

    pattern = r"-?\d+(?:\.\d+)?"

    numbers = re.findall(pattern, text)

    return [float(x) for x in numbers]


# ============================================================
# Direct Match
# ============================================================

def is_direct_match(pred, input_numbers):

    for x in input_numbers:

        if abs(pred - x) <= TOLERANCE:
            return True

    return False
```

```python
# ============================================================
# Division Match
# ============================================================

def is_division_match(pred, input_numbers):

    n = len(input_numbers)

    for i in range(n):

        for j in range(n):

            if i == j:
                continue

            a = input_numbers[i]
            b = input_numbers[j]

            # if abs(b) <= TOLERANCE:
            #     continue

            ratio = a / b

            if abs(pred - ratio) <= TOLERANCE:
                return True

    return False


# ============================================================
# Numerical Correctness
# ============================================================

def numerical_correctness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)

    pred_numbers = extract_numbers(prediction_text)

    direct_match = 0
    division_match = 0
    incorrect = 0

    for pred in pred_numbers:

        # 1. Có trực tiếp trong input
        if is_direct_match(pred, input_numbers):

            direct_match += 1

        # 2. Không có nhưng bằng phép chia
        elif is_division_match(pred, input_numbers):

            division_match += 1

        # 3. Sai hoàn toàn
        else:

            incorrect += 1
```

```
    correct = direct_match + division_match

    if len(pred_numbers) == 0:
        score = 0
    else:
        score = correct / len(pred_numbers)

    return {

        "input_numbers": len(input_numbers),

        "predicted_numbers": len(pred_numbers),

        "direct_match": direct_match,

        "division_match": division_match,

        "correct": correct,

        "incorrect": incorrect,

        "NumericalCorrectness": score
    }


# ============================================================
# Evaluate
# ============================================================

rows = []

for sample in df_csv.to_dict(orient="records"):

    rows.append(

        numerical_correctness(
            sample["input"],
            sample["prediction"]
        )

    )

df = pd.DataFrame(rows)

print(df)

print("="*70)
print("Average")
print("="*70)
print(df.mean(numeric_only=True))

print()

print("="*70)
print("Percentage")
print("="*70)

print(f"Average numbers in Input      : {df['input_numbers'].mean():.2f}")
print(f"Average numbers in Prediction :
{df['predicted_numbers'].mean():.2f}")
```

```
print(f"Direct Match                  : {df['direct_match'].mean():.2f}")
print(f"Division Match                : {df['division_match'].mean():.2f}")
print(f"Correct Numbers               : {df['correct'].mean():.2f}")
print(f"Incorrect Numbers             : {df['incorrect'].mean():.2f}")
print(f"Numerical Correctness         :
{df['NumericalCorrectness'].mean()*100:.2f}%")

df.to_csv(
    f"./Training_Data/NumericalCorrectness_Gemini-3.6-flash.csv",
    index=False
)

print("\nSaved to NumericalCorrectness.csv")
```

```
     input_numbers  predicted_numbers  direct_match  division_match  correct  \
0               16                 37            21               3       24
1                8                  6             5               0        5
2               16                 17            12               4       16
3               16                 15            14               1       15
4                8                 12            10               1       11
..             ...                ...           ...             ...      ...
116             17                 15            12               3       15
117             16                 26            21               4       25
118              8                  6             4               0        4
119              8                  6             4               0        4
120              8                 28            22               0       22

     incorrect  NumericalCorrectness
0           13              0.648649
1            1              0.833333
2            1              0.941176
3            0              1.000000
4            1              0.916667
..         ...                   ...
116          0              1.000000
117          1              0.961538
118          2              0.666667
119          2              0.666667
120          6              0.785714

[121 rows x 7 columns]
================================================================================
Average
================================================================================
input_numbers           12.181818
predicted_numbers       18.842975
direct_match            13.611570
division_match           2.157025
correct                 15.768595
incorrect                3.074380
NumericalCorrectness     0.858684
dtype: float64

================================================================================
Percentage
================================================================================
Average numbers in Input      : 12.18
Average numbers in Prediction : 18.84
Direct Match                  : 13.61
Division Match                : 2.16
Correct Numbers               : 15.77
Incorrect Numbers             : 3.07
```

```
Numerical Correctness         : 85.87%

Saved to NumericalCorrectness.csv
```

In [ ]: